

Android Development: Comprehensive Quiz Study Guide

Section 1: Introduction to Mobile App Development

- **Native App:** Built for one OS (Kotlin/Java for Android). Best performance/hardware access.
- **Web App:** Runs in browser. Low cost, no hardware access.
- **Cross-Platform:** Shared code (Flutter/React Native). Compiles to native-like performance.
- **MADLC:** Idea → Design → Development → Testing → Deployment → Maintenance.

Section 1 MCQs

1. **Which app type offers raw access to hardware?** (A) Web (B) Native (C) Hybrid (D) PWA
-> **Ans: B**
2. **When was the original iPhone released?** (A) 2005 (B) 2007 (C) 2008 (D) 2010 -> **Ans: B**
3. **Android's Q3 2024 market share is approximately:** (A) 27% (B) 50% (C) 72.2% (D) 85%
-> **Ans: C**
4. **Primary language for modern iOS?** (A) Java (B) Objective-C (C) Swift (D) Kotlin -> **Ans: C**
5. **A "Super-app" is most common in:** (A) USA (B) Asia (C) Europe (D) Africa -> **Ans: B**
6. **SRS stands for:** (A) System Reset (B) Software Requirements Specification (C) Simple Storage (D) Secure Run -> **Ans: B**
7. **React Native was developed by:** (A) Google (B) Microsoft (C) Meta (D) Apple -> **Ans: C**
8. **Projected 2023 app revenue?** (A) \$100B (B) \$500B (C) \$935B (D) \$2T -> **Ans: C**
9. **Which is NOT part of MADLC?** (A) Idea (B) Design (C) Compilation (D) Testing -> **Ans: C**
10. **PWAs are mainly promoted by:** (A) Apple (B) Google (C) Meta (D) Amazon -> **Ans: B**

Section 2: Project Architecture & UI

- **Manifest:** Registers components and permissions. Entry point (MAIN/LAUNCHER).
- **Gradle:** Build config (minSdk, targetSdk).
- **dp vs sp:** dp for sizes, sp for text. Baseline is **160dpi**.
- **Layouts:** ConstraintLayout (Flat/Fast), LinearLayout (Rows/Columns), FrameLayout (Stacking).

Section 2 MCQs (Architecture & UI)

1. **First file read by the OS?** (A) build.gradle (B) AndroidManifest.xml (C) R.kt (D) strings.xml
-> **Ans: B**
2. **Unit for textSize?** (A) dp (B) px (C) sp (D) pt -> **Ans: C**
3. **Qualifier for tablets?** (A) layout-land (B) layout-sw600dp (C) values-night (D) drawable-hdpi -> **Ans: B**
4. **ScaleType that crops to fill?** (A) fitCenter (B) centerInside (C) centerCrop (D) fitXY ->

Ans: C

5. **Never commit to Git:** (A) build.gradle (B) settings.gradle (C) local.properties (D) gradle.properties -> **Ans: C**
6. **Flattest hierarchy?** (A) Linear (B) Table (C) Constraint (D) Relative -> **Ans: C**
7. **Dimension for weight?** (A) match_parent (B) wrap_content (C) 0dp (D) 100dp -> **Ans: C**
8. **Unit tests location:** (A) src/main (B) src/test (C) src/androidTest (D) res/raw -> **Ans: B**
9. **android:hint is used in:** (A) TextView (B) Button (C) EditText (D) ProgressBar -> **Ans: C**
10. **Requires RadioGroup:** (A) CheckBox (B) Switch (C) RadioButton (D) Spinner -> **Ans: C**

Section 3: Lifecycles & View Groups

- **Activity Lifecycle:** onCreate → onStart → onResume. Rotation causes Destroy/Recreate.
- **Fragment Lifecycle:** onAttach → onCreateView → onViewCreated. Must null binding in onDestroyView.
- **RecyclerView:** Uses layoutManager + Adapter + ViewHolder. Recycles views to save memory.
- **WebView:** Embeds browser. Needs INTERNET permission.

Section 3 MCQs (Lifecycles & View Groups)

1. **Method after onStart?** (A) onCreate (B) onRestart (C) onResume (D) onPause -> **Ans: C**
2. **Dialog covers activity state?** (A) Stopped (B) Paused (C) Destroyed (D) Killed -> **Ans: B**
3. **RecyclerView mandatory pattern?** (A) ArrayAdapter (B) ViewHolder (C) BaseAdapter (D) SimpleAdapter -> **Ans: B**
4. **Notify only one item change?** (A) notifyDataSetChanged (B) notifyItemChanged (C) notifyAll (D) updateView -> **Ans: B**
5. **Survives screen rotation?** (A) Activity (B) Fragment (C) ViewModel (D) View -> **Ans: C**
6. **Fragment ViewBinding cleanup?** (A) onAttach (B) onCreate (C) onDestroyView (D) onDetach -> **Ans: C**
7. **WebView enables what by default?** (A) JS (B) Zoom (C) Nothing (D) CSS -> **Ans: C**
8. **sequence on Home Button?** (A) onPause->onStop (B) onPause->onDestroy (C) onStop (D) onStop->onDestroy -> **Ans: A**
9. **Sequence on Back Button?** (A) onPause->onStop (B) onPause->onStop->onDestroy (C) onDestroy (D) onPause -> **Ans: B**
10. **Where to initialize UI?** (A) onStart (B) onResume (C) onCreate (D) onAttach -> **Ans: C**

Section 4: OS Architecture & Data Passing

- **Layers:** Linux Kernel → HAL → Native Libs/Runtime → Framework → Apps.
- **Binder:** Kernel driver for Inter-Process Communication (IPC).
- **Zygote:** Forks new app processes for speed.
- **Passing Data:**
 - **Activity → Activity:** Intent.putExtra() / ActivityResultLauncher.
 - **Activity → Fragment:** Bundle via newInstance().

- **Fragment ↔ Fragment:** Shared ViewModel (Activity-scoped).

Section 4 MCQs (OS & Data Passing)

1. **HAL stands for:** (A) High Access Layer (B) Hardware Abstraction Layer (C) Home App Link (D) Hybrid App Logic -> **Ans: B**
2. **Binder IPC driver location?** (A) Framework (B) Linux Kernel (C) Runtime (D) HAL -> **Ans: B**
3. **Zygote process role?** (A) Garbage collection (B) Fast app launch via forking (C) Security (D) Storage -> **Ans: B**
4. **Data class must implement what to be passed in Intent?** (A) Serializable (B) Parcelable (C) List (D) Map -> **Ans: B**
5. **Fragment to Fragment recommended?** (A) Intent (B) Static (C) Shared ViewModel (D) Global -> **Ans: C**
6. **ART compilation type?** (A) JIT (B) AOT (C) Interpreted (D) Scripted -> **Ans: B**
7. **SQLite is located in:** (A) Framework (B) Native Libraries (C) Kernel (D) HAL -> **Ans: B**
8. **App Sandbox is based on:** (A) Java (B) XML (C) Linux User IDs (D) Manifest -> **Ans: C**
9. **Return data from Activity B?** (A) Intent (B) setResult() (C) finish() (D) startActivity() -> **Ans: B**
10. **Fragment arguments use what container?** (A) Intent (B) Bundle (C) List (D) Array -> **Ans: B**

Section 5: Data Persistence

- **SharedPreferences:** Small key-value pairs (XML). Use apply() (async) over commit() (sync).
- **Internal Files:** Private storage (filesDir). Use Gson for JSON serialization.
- **Room Database:** SQLite wrapper. Components: Entity, DAO, Database.
- **DataStore:** Modern async alternative to SharedPreferences.

Section 5 MCQs (Persistence)

1. **Which SharedPreferences method is asynchronous?** (A) commit() (B) apply() (C) save() (D) push() -> **Ans: B**
2. **Room component for SQL queries?** (A) Entity (B) Database (C) DAO (D) Repository -> **Ans: C**
3. **Best for structured relational data?** (A) Files (B) SharedPreferences (C) Room (D) DataStore -> **Ans: C**
4. **Default mode for SharedPreferences?** (A) PRIVATE (B) PUBLIC (C) READABLE (D) APPEND -> **Ans: A**
5. **DataStore is built on what?** (A) Threads (B) Coroutines & Flow (C) AsyncTasks (D) Services -> **Ans: B**
6. **Files stored in filesDir are:** (A) Public (B) Private to the app (C) SD Card (D) Cloud -> **Ans: B**
7. **What happens to SharedPreferences on Uninstall?** (A) Stays (B) Deleted (C) Backed up (D) Hidden -> **Ans: B**

8. **Annotation to mark a primary key in Room?** (A) @ID (B) @Key (C) @PrimaryKey (D) @AutoID -> **Ans: C**
9. **Gson library is used to convert objects to:** (A) XML (B) Binary (C) JSON (D) SQL -> **Ans: C**
10. **Type safety is a benefit of:** (A) Files (B) SharedPreferences (C) DataStore (D) Cache -> **Ans: C**

Section 6: Android Services & Background Work

- **Started Service:** Runs until stopSelf(). START_STICKY restarts if killed.
- **Foreground Service:** Visible to user (Notification). High priority.
- **Bound Service:** Client-server interaction using IBinder.
- **WorkManager:** Guaranteed deferrable work (Network/Battery constraints).

Section 6 MCQs (Services)

1. **Where must all Services be declared?** (A) build.gradle (B) Strings.xml (C) AndroidManifest.xml (D) R.java -> **Ans: C**
2. **Foreground Service requirement?** (A) Hidden (B) Persistent Notification (C) UI screen (D) Bluetooth -> **Ans: B**
3. **Restarts service with null intent?** (A) START_STICKY (B) START_NOT_STICKY (C) START_REDELIVER (D) STOP -> **Ans: A**
4. **Service runs on which thread by default?** (A) Background (B) Main/UI (C) IO (D) Default -> **Ans: B**
5. **WorkManager is best for:** (A) Music (B) Immediate UI (C) Deferrable/Guaranteed work (D) Games -> **Ans: C**
6. **Communication bridge for Bound Services?** (A) Intent (B) IBinder (C) Bundle (D) Notification -> **Ans: B**
7. **Method called when bindService() is used?** (A) onCreate (B) onStart (C) onBind (D) onStop -> **Ans: C**
8. **WorkManager constraint example?** (A) Screen on (B) Wi-Fi Connected (C) Sound on (D) App open -> **Ans: B**
9. **Foreground Service kill priority?** (A) Highest (B) Medium (C) Lowest (D) Random -> **Ans: C** (Less likely to be killed)
10. **Replacement for IntentService?** (A) Activity (B) CoroutineWorker/WorkManager (C) Thread (D) Timer -> **Ans: B**

Section 7: Short & Viva Questions

5 Short Questions

1. **Difference between apply() and commit()?** apply() is asynchronous and returns nothing; commit() is synchronous and returns a boolean success value.
2. **What is a "Memory Leak" in Fragments?** Holding a reference to the ViewBinding after onDestroyView() is called.

3. **What is the Zygote process?** A process that pre-loads Android frameworks and "forks" itself to launch new apps quickly.
4. **When use WorkManager over Service?** Use WorkManager for tasks that must be guaranteed to run even if the app or device restarts, especially if they depend on constraints like Wi-Fi.
5. **Explain START_STICKY.** It tells the OS to recreate the service if it gets killed due to low memory, passing a null intent to onStartCommand.

5 Viva-Style Questions

1. **"Is a Service a separate thread?"** No, it runs on the UI thread by default. You must create a thread/coroutine for heavy work.
2. **"What is the difference between dp and sp?"** dp is for layout, sp is for text and respects system font size changes.
3. **"What is the R class?"** An auto-generated class that contains integer IDs for all resources in the res folder.
4. **"Why use a Repository in Room?"** It provides a clean API for the rest of the app to access data and mediates between different data sources (e.g., local DB and network).
5. **"What does fillViewport do?"** In a ScrollView, it ensures the child view stretches to fill the screen if its content is shorter than the viewport.